



Tutorial >

Controlling time in script

Updated: 2018-09-06

Still there? Have you finished that coffee yet? No? Good. Our component setup is now done. Now, it's time to get our hands dirty with scripting.

Creating the base behavior

The first thing we'll do is create a base behaviour that offers a useful shortcut to use the Timeline component. It's an optional one-time operation, but it will make your code much more legible.

When using Chronos, you will almost always work with the Timeline class, attached as a component to your game objects. It provides all the time measurements that you need, such as `deltaTime` and `timeScale`, and a few more utility methods that we'll cover later on.

Just like you use `transform.position` as a shortcut of `GetComponent<Transform>().position`, you'll be able to use `time.deltaTime` instead of the verbose `GetComponent<Timeline>.deltaTime`.

Create a new script under Scripts called `BaseBehaviour`, and give it the following code:

```
using UnityEngine;
using Chronos;

public class BaseBehaviour : MonoBehaviour
{
    public Timeline time
    {
        get
        {
            return GetComponent<Timeline>();
        }
    }
}
```



From now on, **any script inheriting from `BaseBehaviour`** will have access to the time shortcut property.

Setting the enemy time scale via input

Next, we want to change the time scale of our Enemies clock by pressing the keys 1 through 5 on the keyboard, from rewind to accelerate. Luckily, since we have done our setup correctly, this is a very trivial script.

Create a new script under Scripts called TimeControl, and add give it the following code:

```
using UnityEngine;
using Chronos;

public class TimeControl : MonoBehaviour
{
    void Update()
    {
        // Get the Enemies global clock
        Clock clock = Timekeeper.instance.Clock("Enemies");

        // Change its time scale on key press
        if (Input.GetKeyDown(KeyCode.Alpha1))
        {
            clock.localTimeScale = -1; // Rewind
        }
        else if (Input.GetKeyDown(KeyCode.Alpha2))
        {
            clock.localTimeScale = 0; // Pause
        }
        else if (Input.GetKeyDown(KeyCode.Alpha3))
        {
            clock.localTimeScale = 0.5f; // Slow
        }
        else if (Input.GetKeyDown(KeyCode.Alpha4))
        {
            clock.localTimeScale = 1; // Normal
        }
        else if (Input.GetKeyDown(KeyCode.Alpha5))
        {
            clock.localTimeScale = 2; // Accelerate
        }
    }
}
```



If you want to make the time scale change smoothly, you can use the `LerpTimeScale` method.

Attach this script to any GameObject in the scene. In this case, it would make sense to attach it to the Timekeeper. Now, try playing the game and pressing the keys. You'll notice things don't work quite like they should...

- Accelerating and slowing down time doesn't work
- Pausing time makes enemies ignore gravity

- Rewinding offers some unpredictable behaviour

So, what's wrong? Well, the enemy script is trying to move without any knowledge of Chronos. Luckily, it's an easy fix.

Modifying the enemy script

Chronos will take care of the animations, physics, audio and particle effects for you. However, you still need to make sure your scripts take into consideration the timing measurements that Chronos calculates!

In the platformer demo, the enemy movement is controlled by setting their velocity in the X axis. This occurs in the file Scripts/Enemy at line 48 (line breaks added for legibility):

```
GetComponent<Rigidbody2D>().velocity = new Vector2  
(  
    transform.localScale.x * moveSpeed,  
    GetComponent<Rigidbody2D>().velocity.y  
);
```

We previously used a timeline on our enemy to enable rewinding. The problem here is that the timeline we attached has its own properties and methods for rigidbodies, and therefore becomes confused! We are *bypassing* Chronos by setting the rigidbody's velocity directly. For it to work correctly, we have to tell *the timeline* — not the rigidbody — to change its velocity.

First, we will want to use our time shortcut, so we need to change the base class of the script (and of course, add Chronos as a using):

```
using UnityEngine;  
using System.Collections;  
using Chronos;  
public class Enemy : BaseBehaviour // ...
```

Then, change the component call:

```
// Use Timeline instead of Rigidbody2D  
time.rigidbody2D.velocity = new Vector2  
(  
    transform.localScale.x * moveSpeed,  
    time.rigidbody2D.velocity.y  
);
```



We tried to minimize the amount of code change required to integrate Chronos into a project. However, there are some quirks such as this one that we couldn't avoid.

For a list of all the small changes needed to migrate your script to Chronos, see the [Migration guide](#).

When rewinding, the timeline will interpolate between recorded snapshots and apply them. In that state, the rigidbody isn't really a physical object anymore; it's set to kinematic. For this reason, if we try change its physical properties, Chronos will throw an exception. Therefore, we need to add an if clause around that movement line to verify that time is going forward:

```
if (time.timeScale > 0) // Move only when time is going forward
{
    time.rigidbody2D.velocity = new Vector2
    (
        transform.localScale.x * moveSpeed,
        time.rigidbody2D.velocity.y
    );
}
```

That's it!



Recommended: For more examples of using the Timeline class, see the [API Reference](#).

If you play the game now, enemies should move as expected in regards to time. There is one last detail that's wrong, however: the spawner keeps spawning at the same speed no matter the time scale, and enemies rewound up to the moment of their spawn are not destroyed! Let's fix that.

Modifying the spawner

First, add a timeline component to the spawner prefab (Prefabs/spawner) like we did for the enemies. Set it to observe the global "Enemies" clock as well. Set it to rewindable, because we will need to "despawn" enemies later on.

Open up the spawner script (Scripts/Spawner) in your code editor. It should look like this:

```
using UnityEngine;
using System.Collections;

public class Spawner : MonoBehaviour
{
    public float spawnTime = 5f; // The amount of time between each spawn.
    public float spawnDelay = 3f; // The amount of time before spawning starts.
    public GameObject[] enemies; // Array of enemy prefabs.

    void Start ()
    {
        // Start calling the Spawn function repeatedly after a delay .
        InvokeRepeating("Spawn", spawnDelay, spawnTime);
    }

    void Spawn ()
    {
        // Instantiate a random enemy.
```

```

        int enemyIndex = Random.Range(0, enemies.Length);
        Instantiate(enemies[enemyIndex], transform.position, transform.rotation);

        // Play the spawning effect from all of the particle systems.
        foreach(ParticleSystem p in GetComponentsInChildren<ParticleSystem>())
        {
            p.Play();
        }
    }
}

```

We can see that the spawning timing is done through the `InvokeRepeating` method. Chronos doesn't provide an equivalent for this method out of the box, because a simple coroutine can do the same thing — with more performance and flexibility.

For that reason, let's first change our code to do exactly the same thing, but with a coroutine:

```

using UnityEngine;
using System.Collections;

public class Spawner : MonoBehaviour
{
    public float spawnTime = 5f; // The amount of time between each spawn.
    public float spawnDelay = 3f; // The amount of time before spawning starts.
    public GameObject[] enemies; // Array of enemy prefabs.

    void Start ()
    {
        // Start calling the Spawn coroutine.
        StartCoroutine(Spawn());
    }

    IEnumerator Spawn ()
    {
        yield return new WaitForSeconds(spawnDelay); // Wait for the delay

        while (true) // Repeat infinitely
        {
            // Instantiate a random enemy.
            int enemyIndex = Random.Range(0, enemies.Length);
            Instantiate(enemies[enemyIndex], transform.position, transform.rotation);

            // Play the spawning effect from all of the particle systems.
            foreach(ParticleSystem p in GetComponentsInChildren<ParticleSystem>())
            {
                p.Play();
            }

            // Wait for the interval
            yield return new WaitForSeconds(spawnTime);
        }
    }
}

```

At this point, Chronos still isn't passing timing calculations to the spawner. That's because we're using Unity's `WaitForSeconds` method, which only respects Unity's `Time.timeScale`, a value we haven't

even touched. That means our spawner will execute at normal speed no matter what time scale we assign to our Enemies clock.

Fortunately, converting any coroutine to Chronos is easy. Simply replace new `WaitForSeconds()` by `Timeline.WaitForSeconds`. No further work is needed. The updated version of our spawner component is below (notice the using and the base behaviour):

```
using UnityEngine;
using System.Collections;
using Chronos;

public class Spawner : BaseBehaviour
{
    public float spawnTime = 5f; // The amount of time between each spawn.
    public float spawnDelay = 3f; // The amount of time before spawning starts.
    public GameObject[] enemies; // Array of enemy prefabs.

    void Start ()
    {
        // Start calling the Spawn coroutine.
        StartCoroutine(Spawn());
    }

    IEnumerator Spawn ()
    {
        yield return time.WaitForSeconds(spawnDelay); // Wait for the delay

        while (true) // Repeat infinitely
        {
            // Instantiate a random enemy.
            int enemyIndex = Random.Range(0, enemies.Length);
            Instantiate(enemies[enemyIndex], transform.position, transform.rotation);

            // Play the spawning effect from all of the particle systems.
            foreach(ParticleSystem p in GetComponentsInChildren<ParticleSystem>())
            {
                p.Play();
            }

            // Wait for the interval
            yield return time.WaitForSeconds(spawnTime);
        }
    }
}
```

For our spawner to be coherent in time, the last thing we need to do is make it able to despawn enemies once they're rewound up to the moment of their instantiation. Chronos makes this process simple with a concept called Occurrences.

[Previous](#)

[← Adding timelines](#)

Next

[Using occurrences](#) →

Was this article helpful?

Be the first to vote!

👍 Yes, helpful

👎 No, not for me



PRODUCTS

LUDIQ

Contact

Blog

Copyright © 2022 Ludiq. All Rights Reserved. • • Terms of Service • Icons by IconMonstr and FatCow